# Project - High Level Design

# On

# IaC Provisioning for Telecomm System

Course Name: Devops

***Institution Name:*** Medicaps University – Datagami Skill Based Course

*Student Name(s) & Enrolment Number(s):*

| Sr no | Student Name | Enrolment Number |
|---|---|---|
| 1 | ANKIT YADAV | EN22CS301136 |
| 2 | ANSHUMAN GEETE | EN22CS301160 |
| 3 | ARTI JAIN | EN22CS301206 |
| 4 | ATISHA JAIN | EN22CS301235 |
| 5 | AVANI SHARMA | EN22CS301239 |
| 6 | ISHANA HATODIYA | EN23CS3L1011 |

*Group Name: 07D2*

*Project Number: DO-19*

*Industry Mentor Name:*

*University Mentor Name: Prof. Akshay Saxena*

*Academic Year: 2022-2026*

## Table of Contents

# 1. Introduction

The project demonstrates the automation of infrastructure provisioning for a Telecom-grade environment using Infrastructure as Code (IaC) tools including Terraform and Ansible.

The objective is to replace manual infrastructure setup with automated, repeatable, error-free provisioning that can deploy both local Docker-based telecom service environments and cloud infrastructure (AWS). The system provisions compute instances, installs dependencies, configures telecom network components, and deploys required services using automated scripts.

## 1.1 Scope of the Document

This document covers:

- Complete IaC system architecture and provisioning workflow
- Terraform configuration for cloud resource provisioning
- Ansible playbooks for configuration management
- Docker-based local telecom environment deployment
- CI/CD automation for provisioning
- Security considerations and environment variable handling
- API catalogue (if applicable for telecom services)

Excluded:

- Internal telecom business logic
- Proprietary telecom signalling algorithms
- Network protocol handling of telecom core elements

## 1.2 Intended Audience

| Audience | Purpose |
|---|---|
| DevOps Engineers | Understand IaC automation using Terraform and Ansible |
| Cloud Architects | Review provisioning, security, and scaling considerations |
| Telecom Engineers | Understand automated telecom environment deployment |
| Academic Evaluators | Technical depth and implementation analysis |
| Future Maintainers | Extend provisioning scripts safely |

A basic understanding of Terraform, Ansible, AWS, and Docker is recommended.

## 1.3 System Overview

The system consists of two decoupled repositories:

**Repository 1 — IaC Scripts (Telecomm-IaC-Provisioning)**

Contains Terraform modules, Ansible playbooks, inventory files, Docker environment setup, and automation scripts.

**Repository 2 — Telecom Application Repository**

Contains telecom simulation services (e.g., Call Simulation, SMS Router, Signalling Handler, Network Registry).

**Workflow Overview**

Whenever infrastructure provisioning is triggered:

1. Terraform provisions cloud infrastructure or local Docker networks.
2. Ansible configures OS, installs dependencies, deploys telecom services.
3. Docker containers simulate telecom components locally or on EC2.
4. CI/CD automates the entire provisioning pipeline.

# 2. System Design

## 2.1 Application Design

The system provisions two environments:

- Cloud Environment (AWS) via Terraform
- Local Docker Telecom Network via Docker Compo

Telecom Components in Application Repo

| **Module** | **Purpose** |
|---|---|
| Call Handler | Simulates call signalling |
| SMS Router | Routes messages |
| Network Registry | Stores subscriber metadata |
| Monitoring Dashboard | Streamlit/Grafana-based UI |

## 2.2 Process Flow

The process begins when a developer writes code and pushes it to the GitHub repository. This action triggers the CI/CD pipeline configured using Jenkins or GitHub Actions. The pipeline pulls the latest code and performs the build process by installing dependencies and preparing the application. After the build, automated testing is executed to ensure code quality. If the tests pass successfully, a Docker image is created using a Dockerfile and pushed to a Docker registry.AWS then pulls this image and deploys it as containers within the cluster. Finally, the application becomes accessible to users through AWS services.

## 2.3 Information Flow

The system follows a structured flow of information:

- User → Frontend (input collection)
- Frontend → Backend API (/chat)
- Backend → AI API (request processing)
- AI API → Backend (response generation)
- Backend → Frontend (response delivery)
- Frontend → User (display output)

This ensures efficient data exchange and minimal latency

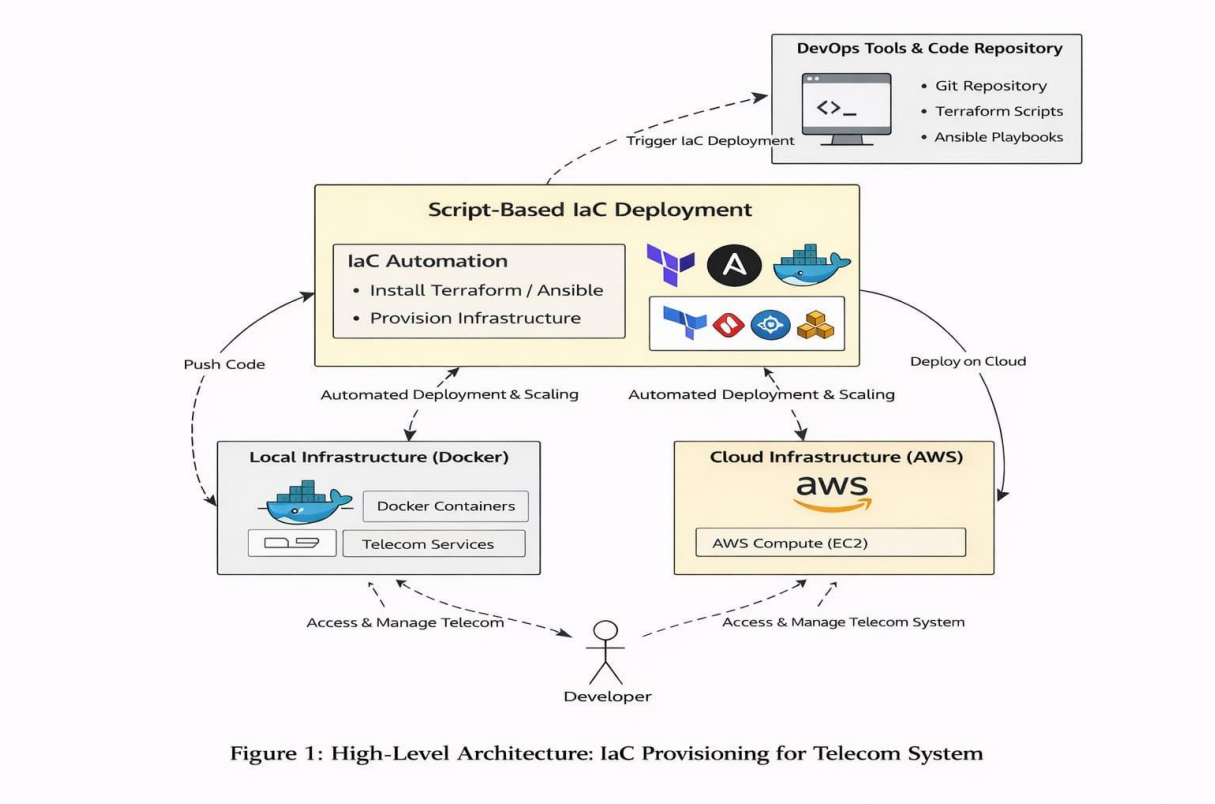


Figure 1: High-Level Architecture: IaC Provisioning for Telecom System

## 2.4 Components Design

### Component 1 — Terraform (Infrastructure Provisioning)

| Property | Value |
|---|---|
| Provider | AWS |
| Region | ap-south-1 |
| Instance Type | t3.small |
| VPC | Custom or Default |
| Security Group | SIP + HTTP + SSH |
| AMI | Amazon Linux 2 |

Key Terraform Steps

- Initialize provider
- Create VPC + Subnets
- Create Security Groups
- Create EC2 Instances
- Output public/private IPs

### Component 2 — Ansible (Configuration Management)

| Configuration | Value |
|---|---|
| Inventory Source | Terraform output |
| Playbooks | install-docker.yml, deploy-services.yml |
| SSH Access | Private key from GitHub Secrets |

Responsibilities

- Install Docker & dependencies
- Pull telecom service containers
- Start telecom network services
- Setup monitoring stack

### Component 3 — Local Docker Telecom Network

| Component | Port | Purpose |
|---|---|---|
| SIP Handler | 5060 | Call signalling |
| SMS Router | 8081 | Message routing |
| Registry Service | 9000 | Subscriber DB |
| Monitoring | 3000 | Grafana/Streamlit |

Docker compose provisions:

- Telecom containers
- Shared overlay network
- Volume mounts
- Health checks

### Component 4 — GitHub Actions (Automation Pipeline)

| Step | Action |
|---|---|
| 1 | Checkout Repo |
| 2 | Install Terraform & Ansible |
| 3 | Run Terraform Init/Plan/Apply |
| 4 | Parse EC2 outputs |
| 5 | Run Ansible Playbooks |
| 6 | Validate services are running |

### Component 5 — AWS Infrastructure

| **Component** | **Details** |
|---|---|
| EC2 | t3.small, 2 instances |
| VPC | 1 public subnet, IGW |
| Security Group | SIP/HTTP/SSH |
| IAM | Least privilege role |

## 2.5 Key Design Considerations

| **API** | **Method** | **Purpose** | **Auth** |
|---|---|---|---|
| /call/initiate | POST | Simulate call setup | Token |
| /sms/send | POST | Send SMS routing job | Token |
| /registry/query | GET | Subscriber info | None |
| /health | GET | Service health | None |

Example Response:
200 OK
"status: healthy"

## 2.6 API Catalogue

Sample Request

```
{
 "message": "My internet is not working"
}
```

Sample Response

```
{
 "response": "Please check your data connection and restart your device."
}
```

# 3. Data Design

The data design for the **IaC Provisioning for Telecomm System** focuses on managing configuration, infrastructure state, deployment artifacts, and operational logs across the system lifecycle.

The system does **not rely on heavy business databases**, but instead manages **infrastructure-centric data**, which is critical for automation, reproducibility, and monitoring.

## 3.1 Data Model

The system consists of the following core entities:

1. Infrastructure Configuration

- Represents IaC scripts written in Terraform or Ansible
- Stored as .tf, .yml, or .yaml files
- Defines:
  - Compute resources (EC2 instances)
  - Networking (VPC, Subnets)
  - Security groups
  - Load balancers

Example fields:

- config_id

- resource_type
- region
- instance_type
- variables

## 2. Terraform State File

- Maintains the current state of infrastructure
- Stored locally or remotely (S3 backend)

Contains:

- Resource IDs
- Current configuration snapshot
- Dependency graph

## 3. CI/CD Pipeline Data

- Stores pipeline execution details from GitHub Actions

Attributes:

- pipeline_id
- build_status (success/failure)
- timestamps
- logs
- artifact references

## 4. Docker Image Metadata

- Represents container images built using Docker

Fields:

- image_name
- version/tag
- repository (Docker Hub)
- build_time

## 5. Deployment Instance

- Represents infrastructure instances on Amazon Web Services

Fields:

- instance_id
- IP address
- status (running/stopped)
- region

## 6. Logs and Monitoring Data

- Captures:
  - CI/CD logs
  - Infrastructure logs
  - Application logs

Used for:

- Debugging
- Performance monitoring
- Failure analysis

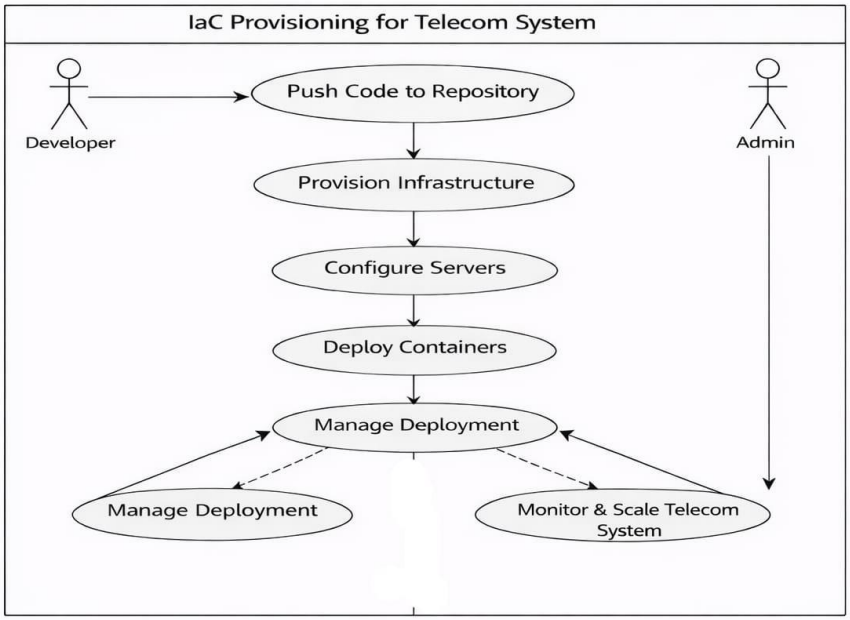


Figure 2: Use Case Diagram - IaC Provisioning for Telecom System

## 3.2 Data Access Mechanism

1. GitHub access is controlled using authentication tokens for secure code management.
2. Docker registry requires secure credentials to pull and push container images.
3. Amazon Web Services resources are accessed using IAM roles and permissions.
4. Terraform state files are securely stored and accessed via remote backends (e.g., S3).
5. Environment variables and secrets are managed securely using encrypted storage mechanisms.

## 3.3 Data Retention Policies

1. CI/CD pipeline logs are stored for monitoring and debugging purposes.
2. Docker images are versioned to support rollback and reproducibility.
3. AWS maintains deployment logs and instance-related data for auditing.
4. Terraform state files are version-controlled to track infrastructure changes.
5. Old logs and unused images may be archived or removed based on retention policies.

### 3.4 Data Migration

1. New application versions generate updated Docker images.
2. AWS deploys updated containers on EC2 instances or Kubernetes clusters.
3. Infrastructure updates are applied using Terraform scripts without manual intervention.
4. Rolling updates or blue-green deployments ensure minimal downtime.
5. Previous versions of the application and infrastructure can be restored if needed.

## 4. Interfaces

4.1 USER INTERFACE

- GitHub Dashboard
- AWS Management Console
- Application UI

4.2 COMMAND LINE INTERFACE

- Terraform CLI
- Ansible CLI
- Docker CLI
- AWS CLI

## 5. State and Session Management

- Infrastructure state managed via Terraform state files
- Application sessions handled within containers
- Cloud instances maintain runtime state

## 6. Caching

- Docker layer caching
- Dependency caching in CI/CD
- Infrastructure reuse via modules

# 7. Functional & Non-Functional Requirements

## 7.1 Functional Requirements

- **Automated Tool Installation:** The system must install Terraform/Ansible automatically if not present.
- **Multi-Platform Support:** Capability to deploy on both AWS and local Docker.
- **Resource Tagging:** Automatically tag all provisioned resources for Telecomm billing and tracking.

## 7.2 Non-Functional Requirements

- **Idempotency:** Repeated execution of scripts must result in the same infrastructure state without errors.
- **Security:** Use of IAM roles and encrypted state files to protect cloud access.
- **Scalability:** Ability to scale the number of Telecomm nodes by changing a single variable.

# 8. References


- Terraform Docs — developer.hashicorp.com
- Ansible Docs — docs.ansible.com
- Docker Docs — docs.docker.com
- AWS EKS/EC2 — docs.aws.amazon.com
- Telecom Standards — 3GPP TS Series


# 9. GitHub

https://github.com/atisha224/IaC-Provisioning-for-Telecomm-System